

Weekly Spotify Wrapped

M122E Final Documentation

Python automation project for a weekly personal music report

Student	Santiago Voutat
Module	M122E - Automate process with a scripting language
School	GIBZ
Teacher	Roger Mueller
Project	Weekly Spotify Wrapped
Submission date	Saturday, 20 June 2026



Last.fm listening history, Spotify metadata, SQLite storage and Gmail delivery in one automated weekly workflow.

Table of Contents

1. Project Overview	3
1.1 Context	3
1.2 Goal	3
1.3 What The Program Does	3
2. Project Work And Personal Approach	3
2.1 How I Started	3
2.2 Difficulties During The Project	3
2.3 What I Improved Near The End	3
3. Project Plan	4
4. Requirements	4
4.1 Functional Requirements	4
4.2 Non-Functional Requirements	4
5. System Design	4
5.1 Module Overview	4
5.2 Main Workflow	5
5.3 System Architecture Diagram	6
5.4 UML Activity Diagram	7
5.5 UML Component Diagram	8
5.6 UML Deployment Diagram	8
5.7 UML Class Diagram	9
5.8 ER Diagram	9
5.9 Flowchart	10
6. Database Design	10
7. Installation And Usage	11
7.1 Prerequisites	11
7.2 Clone Or Open The Project	11
7.3 Install Dependencies	11
7.4 Configure The .env File	11
7.5 API Setup Explained Simply	11
7.6 Start The Program	13
7.7 Weekly Automation	13
8. Testing	13
9. Assignment Coverage	15
10. Conclusion	15

1. Project Overview

1.1 Context

Weekly Spotify Wrapped is my M122E Python automation project. The idea is based on a simple personal problem: I listen to music every week, but I do not want to manually count my top songs, compare artists or create a report myself. The program solves this by collecting my Last.fm listening history, enriching it with Spotify information and turning it into a weekly report.

The project uses Last.fm as the main source for listening data because it provides recent scrobbles with track names, artists, albums and timestamps. Spotify is used as an enrichment API, mainly for cover images, durations and links. SQLite stores the listening history locally, and Gmail SMTP sends the finished report by email.

1.2 Goal

The goal of the project is to automate a complete weekly workflow. Instead of opening several services manually, the script loads data, processes it, stores it, creates statistics, generates a report and sends the result. The final result should feel like a small weekly Spotify Wrapped, but it should be generated automatically and be understandable from the code and documentation.

1.3 What The Program Does

The program starts with configuration loading and ends with a finished report. Between those steps it talks to external APIs, stores data in a database and handles errors in a controlled way. The most important result is the weekly HTML report, but the stored database history is also important because it allows the report to compare the current week with the previous week.

2. Project Work And Personal Approach

2.1 How I Started

I started by reading the assignment requirements and checking what the project proposal still needed. The first version of the idea was mainly about a weekly music report. During planning I made the requirements more concrete, for example by defining that the script should use environment variables, store data in SQLite, prevent duplicates, include tests and handle external API errors.

After the requirements were clearer, I split the project into modules. This helped me keep the code understandable. Each file has one main responsibility, such as loading configuration, calling Last.fm, calling Spotify, writing to the database, generating the report or sending the email.

2.2 Difficulties During The Project

The most difficult part was not one single line of code, but the combination of several external systems. Last.fm, Spotify and Gmail all behave differently. Last.fm is used for the listening history, Spotify needs OAuth for playlist features, and Gmail requires an app password instead of the normal account password.

Another difficulty was the Spotify playlist feature. The report and metadata enrichment worked, but playlist updates can fail with HTTP 403 if the permissions are not correct. I decided not to let this stop the whole program. The playlist update is now optional, and if it fails the report and email workflow can still finish. This was an important reliability decision.

The email design also needed several improvements. At first it was more basic, then I changed it to a darker Spotify-style layout, added cover images and adjusted spacing so the result looked better on desktop and mobile.

2.3 What I Improved Near The End

Near the end of the project I improved the documentation and diagrams because the assignment does not only grade the code. I added a clear architecture diagram, cleaned the final documentation, created PDFs and made the README easier to follow with numbered setup steps. This makes the project easier to understand for someone who opens the GitHub repository for the first time.

3. Project Plan

The project follows the waterfall model from the assignment. I used the phases to keep the work structured, even though I still improved some details later when I tested the real workflow.

Phase	Main Work	Result
Requirements Analysis	Clarify the project idea and define functional and non-functional requirements.	The requirements were made measurable and prioritized.
Design	Plan modules, workflow, database and external systems.	The architecture, UML diagrams, flowchart and ER diagram were created.
Development	Implement configuration, APIs, SQLite, report generation and email sending.	A working modular Python application was completed.
Testing	Write unit tests and run live checks with real services.	35 pytest tests pass and the email workflow was verified.
Documentation	Create final documentation, test protocol, proposal update and README.	The docs folder contains DOCX and PDF files for submission.
Deployment	Plan weekly execution with Windows Task Scheduler.	The automation setup is documented in the README and final documentation.

4. Requirements

4.1 Functional Requirements

ID	Priority	Requirement	Implementation	Status
FR-01	Must	Load configuration values from .env.	config.py	Fulfilled
FR-02	Must	Validate required settings and show clear errors.	config.py and main.py	Fulfilled
FR-03	Must	Fetch Last.fm tracks from the last 7 days.	lastfm_client.py	Fulfilled
FR-04	Must	Store listening data in SQLite.	database.py	Fulfilled
FR-05	Must	Prevent duplicate track plays.	SQLite UNIQUE rule	Fulfilled
FR-06	Must	Calculate top songs, top artists, track count and listening time.	report_generator.py	Fulfilled
FR-07	Must	Generate an HTML report.	report_generator.py	Fulfilled
FR-08	Must	Send the weekly report by Gmail SMTP.	email_sender.py	Fulfilled
FR-09	Should	Enrich tracks with Spotify metadata.	spotify_client.py	Fulfilled
FR-10	Should	Continue if Spotify enrichment or playlist update fails.	main.py	Fulfilled
FR-11	Should	Compare the current week with the previous week.	SQLite history and report_generator.py	Fulfilled
FR-12	Could	Create or update a Spotify playlist.	spotify_setup.py and spotify_client.py	Optional feature

4.2 Non-Functional Requirements

ID	Category	Requirement	Implementation	Status
NFR-01	Security	Secrets must not be hardcoded.	.env, .env.example and .gitignore	Fulfilled
NFR-02	Reliability	API and email errors must be handled clearly.	custom exceptions and logging	Fulfilled
NFR-03	Maintainability	The code should be modular and easy to explain.	separate modules by responsibility	Fulfilled
NFR-04	Testability	Important logic should be covered by pytest.	tests folder	Fulfilled
NFR-05	Usability	Installation and start commands should be understandable.	README and Section 7	Fulfilled
NFR-06	Portability	The project should run on a normal Windows Python setup.	requirements.txt and PowerShell setup	Fulfilled

5. System Design

5.1 Module Overview

The program is split into small modules. This structure makes the workflow easier to understand and also makes the code easier to test. The main file coordinates the work, while the other files focus on one clear task.

Module	Responsibility
main.py	Starts the program and coordinates the complete workflow.
config.py	Loads and validates environment variables.
lastfm_client.py	Loads recent tracks from Last.fm.

spotify_client.py	Adds Spotify metadata and optionally updates a playlist.
database.py	Creates and manages the SQLite database.
report_generator.py	Calculates statistics and creates the HTML report.
email_sender.py	Sends the report through Gmail SMTP.
spotify_setup.py	Helps create Spotify refresh token and playlist values.

5.2 Main Workflow

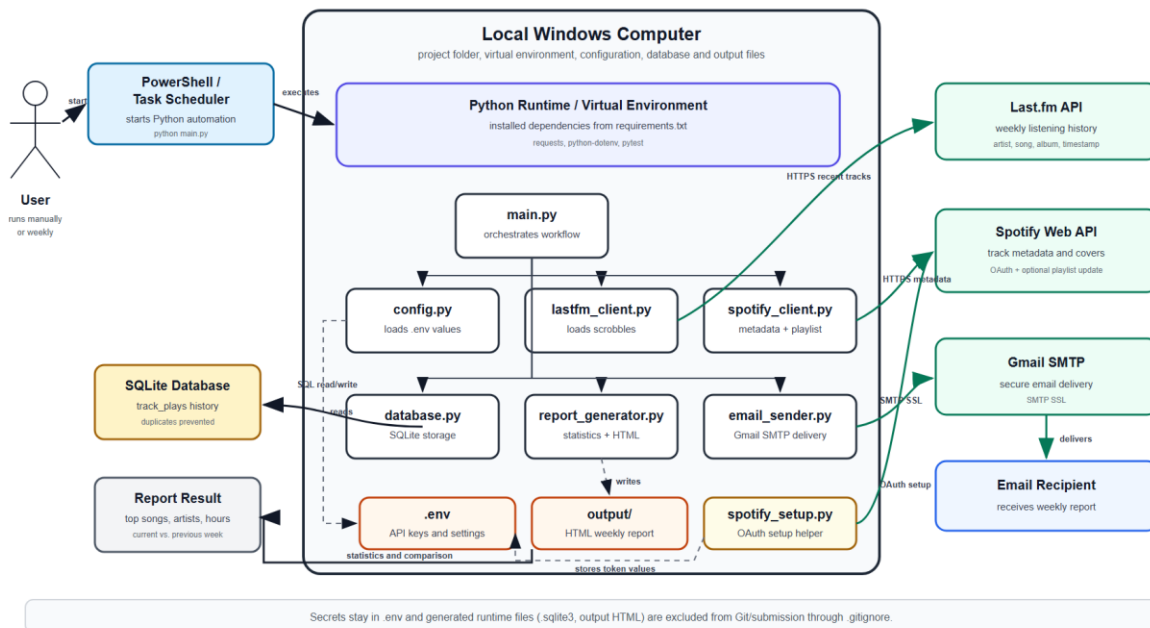
The workflow is linear and easy to follow. The script first prepares the configuration and database, then loads and enriches data, and finally creates and sends the report.

1. The user or Windows Task Scheduler starts python main.py.
2. The configuration is loaded from .env.
3. The SQLite database is initialized.
4. Last.fm tracks are fetched for the configured period.
5. Spotify metadata is added if Spotify credentials are available.
6. Track plays are saved and duplicates are skipped.
7. Current and previous week statistics are calculated.
8. The HTML report is generated and saved in output/.
9. The report is sent by email if SEND_EMAIL is enabled.

5.3 System Architecture Diagram

System Architecture Diagram - Weekly Spotify Wrapped

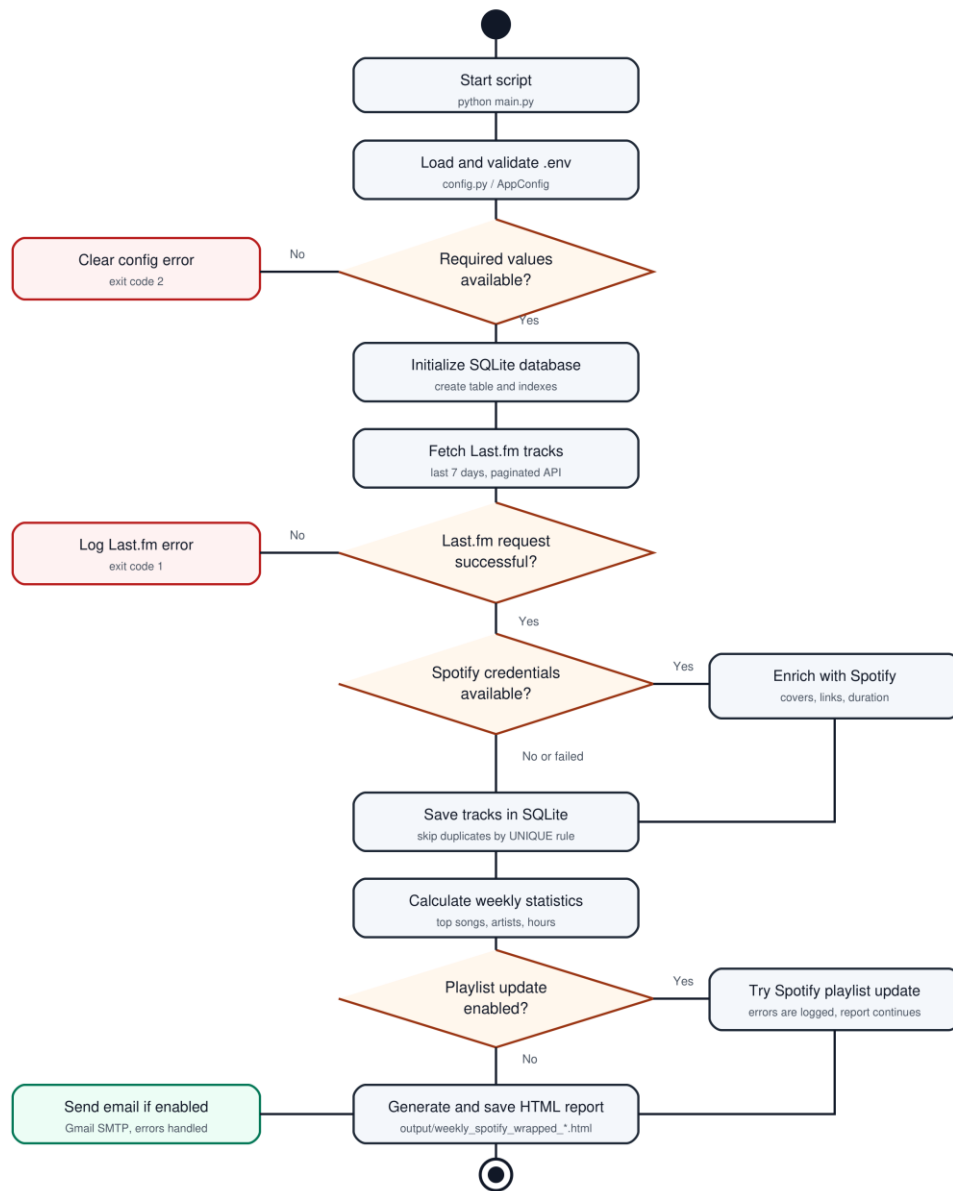
Combined component and deployment view for the M122E automation script



The system architecture diagram gives the best overview of the complete project. It shows the local Windows computer, the Python modules, the .env file, the SQLite database, the output folder and the external services.

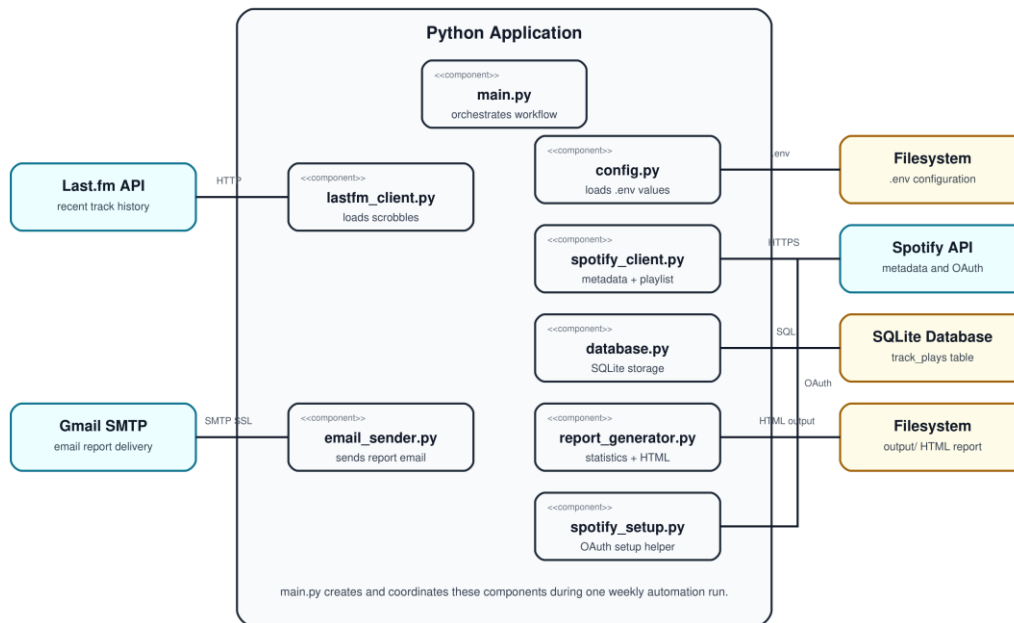
5.4 UML Activity Diagram

UML Activity Diagram - Weekly Spotify Wrapped



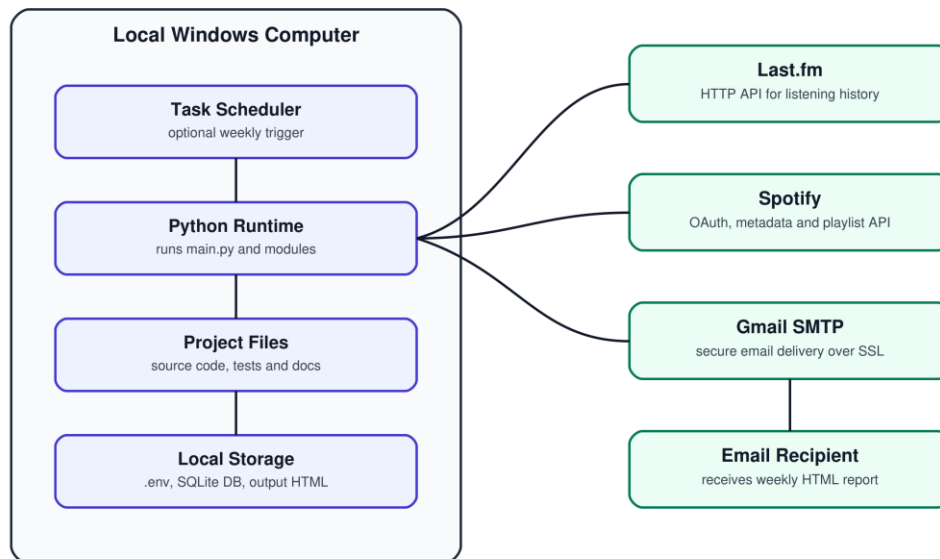
5.5 UML Component Diagram

UML Component Diagram - Weekly Spotify Wrapped



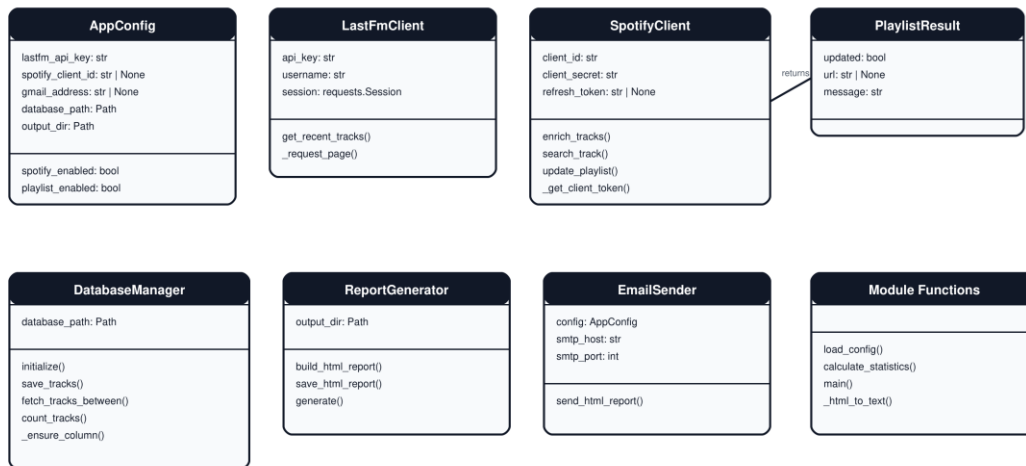
5.6 UML Deployment Diagram

UML Deployment Diagram - Weekly Spotify Wrapped



5.7 UML Class Diagram

UML Class Diagram - Main Python Structures



main.py creates and coordinates the clients, database manager, report generator and email sender.

5.8 ER Diagram

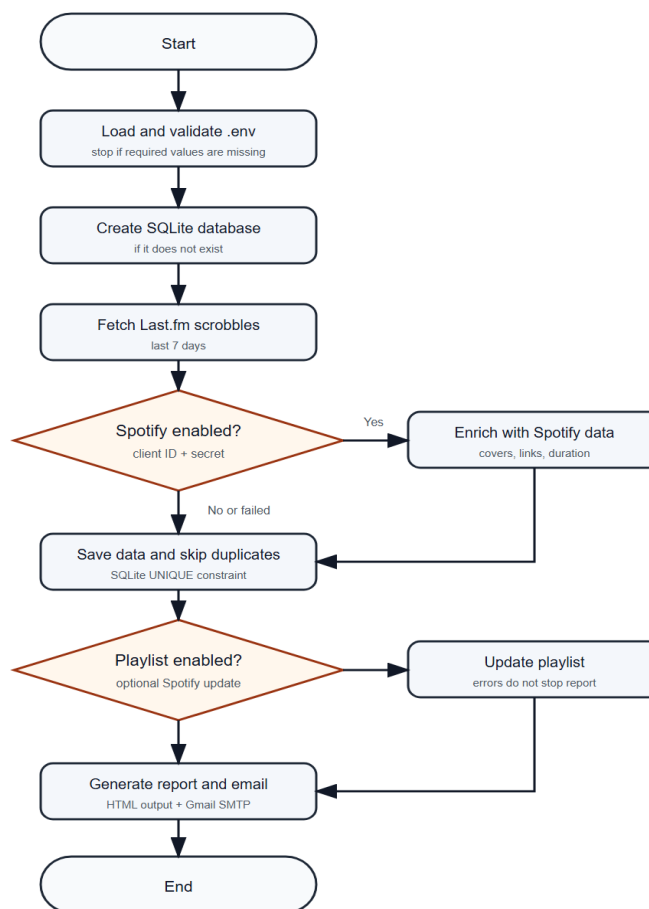
ER Diagram - SQLite Database



Unique constraint: track_name + artist_name + played_at prevents duplicate track plays.

5.9 Flowchart

Flowchart - Weekly Report Automation



6. Database Design

The database is used for more than just temporary storage. It keeps a local listening history so repeated runs do not create duplicate entries and so the current week can be compared with the previous week. This makes the report more useful because it can show changes over time.

Column Or Group	Purpose
track_name, artist_name, album_name	Basic music information from Last.fm and Spotify.
played_at	Timestamp of the track play. This is also used for weekly filtering.
lastfm_url	Optional link to the Last.fm track page.
spotify_track_id and spotify_track_url	Spotify identification and link to the track.
spotify_duration_ms	Duration used to estimate listening time.
spotify_cover_url and spotify_artist_image_url	Images used in the HTML email report.
created_at	Timestamp for when the row was inserted locally.

Duplicate plays are prevented with this database rule:

```
UNIQUE(track_name, artist_name, played_at)
```

7. Installation And Usage

7.1 Prerequisites

The project is designed for a normal Windows setup with PowerShell. Before starting, Python should be installed and the user should have accounts for Last.fm, Spotify and Gmail if all features are used.

Requirement	Why It Is Needed
Python	Runs the automation script.
PowerShell	Used for the setup and start commands.
Last.fm API key	Required to load recent listening data.
Spotify Developer app	Used for metadata and optional playlist updates.
Gmail app password	Required only when email sending is enabled.

7.2 Clone Or Open The Project

If the project is opened from GitHub, clone the repository first.

```
git clone https://github.com/Svoutat/weekly-spotify-wrapped.git
cd weekly-spotify-wrapped
```

If the project is already stored locally, open PowerShell in the existing project folder instead.

```
cd <project-path>
```

7.3 Install Dependencies

The virtual environment keeps the project dependencies separate from other Python projects.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

7.4 Configure The .env File

The .env file contains all private values and settings. The real .env file is ignored by Git and must not be uploaded. The project includes .env.example so a new user can see which values are needed.

```
Copy-Item .env.example .env
```

Value	Explanation
LASTFM_API_KEY	API key from the Last.fm API account page.
LASTFM_USERNAME	The Last.fm username whose scrobbles are loaded.
SPOTIFY_CLIENT_ID and SPOTIFY_CLIENT_SECRET	Used to call the Spotify Web API.
SPOTIFY_REFRESH_TOKEN, SPOTIFY_PLAYLIST_ID and PLAYLIST_URL	Used only for optional playlist updates.
ENABLE_PLAYLIST	Set to true only if Spotify playlist permissions work.
GMAIL_ADDRESS, GMAIL_APP_PASSWORD and RECIPIENT_EMAIL	Required when SEND_EMAIL is true.
SEND_EMAIL	Controls whether the report is emailed or only saved locally.
LOOKBACK_DAYS, DATABASE_PATH and OUTPUT_DIR	Optional settings with useful defaults.

7.5 API Setup Explained Simply

Last.fm setup (required)

Last.fm is required because it provides the listening history.

1. Open <https://www.last.fm/api/account/create>.
2. Log in with a Last.fm account.
3. Create a new API application.

4. Use a clear application name, for example Weekly Spotify Wrapped.
5. Use a short description such as Weekly listening report for a school automation project.
6. Callback URL and homepage are not important for this project. If Last.fm asks for a URL, use `http://localhost` as a placeholder.
7. Copy the generated API key into `LASTFM_API_KEY`.
8. Put the Last.fm username whose scrobbles should be loaded into `LASTFM_USERNAME`.
9. The Last.fm API secret is not needed because the script only reads recent public scrobbles.

Spotify setup (recommended, but optional)

Spotify adds album covers, artist images, durations and track links. According to Spotify's Web API Getting Started page, a Spotify Premium account can be required to use the Web API.

1. If the teacher has Spotify Premium or a working Spotify Developer account, open <https://developer.spotify.com/dashboard>.
2. Log in and click Create app.
3. Use an app name such as Weekly Spotify Wrapped.
4. Use a description such as Weekly music report using Spotify metadata.
5. Add `http://127.0.0.1:8888/callback` as redirect URI. This value must match `SPOTIFY_REDIRECT_URI` in the `.env` file.
6. Select Web API as the API or SDK.
7. Accept the Spotify developer terms and create the app.
8. Open the app settings.
9. Copy the client ID into `SPOTIFY_CLIENT_ID`.
10. Reveal the client secret and copy it into `SPOTIFY_CLIENT_SECRET`.
11. The report can already use Spotify metadata with only client ID and client secret. Playlist permissions are not required for the main report.

Alternative without Spotify Premium

If the teacher does not have Spotify Premium or does not want to create a Spotify app, the project can still be tested without Spotify.

1. Leave `SPOTIFY_CLIENT_ID` empty.
2. Leave `SPOTIFY_CLIENT_SECRET` empty.
3. Leave `SPOTIFY_REFRESH_TOKEN` empty.
4. Leave `SPOTIFY_PLAYLIST_ID` empty.
5. Leave `PLAYLIST_URL` empty.
6. Set `ENABLE_PLAYLIST=false`.

In this mode, the program uses Last.fm data only. The database, statistics, HTML report, tests and optional email sending still work. Spotify covers, links, durations and playlist updates are skipped.

Spotify playlist setup (optional)

Playlist updates are not required for the main automation.

1. Only use this part if Spotify works and playlist updates should be tested.
2. Set `ENABLE_PLAYLIST=true`.

3. Run `python spotify_setup.py`.
4. The helper opens a Spotify login URL.
5. After login, Spotify redirects to the local callback URL.
6. The helper saves `SPOTIFY_REFRESH_TOKEN`, `SPOTIFY_PLAYLIST_ID` and `PLAYLIST_URL` in the `.env` file.
7. If Spotify returns HTTP 403, set `ENABLE_PLAYLIST=false`. The report still works without playlist updates.

Gmail setup (optional)

Email sending is optional. If the teacher only wants to test report generation, use `SEND_EMAIL=false`. Then `GMAIL_ADDRESS`, `GMAIL_APP_PASSWORD` and `RECIPIENT_EMAIL` can stay empty, and the script still creates the HTML report in `output/`.

To test email delivery:

1. Use a Gmail account as the sender.
2. Enable 2-Step Verification in the Google account.
3. Open <https://myaccount.google.com/apppasswords>.
4. Create an app password for this project.
5. Put the Gmail address into `GMAIL_ADDRESS`.
6. Put the generated app password into `GMAIL_APP_PASSWORD`.
7. Put the receiver email address into `RECIPIENT_EMAIL`.
8. Set `SEND_EMAIL=true`.
9. Use the app password, not the normal Gmail password.

7.6 Start The Program

After the `.env` file is filled, the program can be started with two commands.

```
.\.venv\Scripts\Activate.ps1
python main.py
```

A successful run creates or updates the SQLite database, loads Last.fm tracks, enriches them with Spotify when possible, saves the HTML report in `output/` and sends the email if `SEND_EMAIL` is true.

7.7 Weekly Automation

The script can be run automatically once per week with Windows Task Scheduler. This is optional, but it matches the goal of the project because the weekly report should not need to be started manually every time.

```
Program:
<project-path>\.venv\Scripts\python.exe

Arguments:
main.py

Start in:
<project-path>
```

8. Testing

Testing was important because the project uses several external systems. The unit tests do not depend on real API calls. Instead, the external services are mocked where needed, which makes the tests faster and safer to run.

```
python -m pytest
```

```
37 passed
```

The tests cover configuration loading, missing values, database duplicate handling, Last.fm parsing, Spotify enrichment, report generation, email sending, Spotify setup and the main workflow. The full executed test protocol is included as M122E_Test_Protocol.pdf and M122E_Test_Protocol.docx.

The repository also includes a GitHub Actions CI pipeline in .github/workflows/ci.yml. The pipeline runs on pushes and pull requests to main, sets up Python 3.12, installs the dependencies from requirements.txt and runs `python -m pytest`. There is no production deployment for this school project, so the CD part is limited to keeping the repository automatically tested and ready for submission.

For the test design, I focused on the parts that could fail in real use: missing configuration values, duplicate listening entries, API responses with incomplete data, email sending and the main workflow. External services are mocked in the unit tests, so the tests can run without using private API keys or sending real emails.

The result is not only that the current version works on my computer, but also that another person can run the tests and see which parts of the automation were checked. This was important for me because the project combines local files, a database and several online services, and problems in one part should be visible quickly.

9. Assignment Coverage

The assignment requires a working script and documentation. The following table shows where the important requirements are covered in this project.

Assignment Requirement	Evidence In This Project
Script performs multiple actions	The workflow loads data, enriches it, stores it, calculates statistics, creates a report and sends email.
Well-organized scripts	The project is split into focused Python modules.
Environment variables	.env, .env.example and config.py are used.
requirements.txt	The dependency file is included in the project root.
Clear start command	The start command is python main.py.
Project plan	The project plan is included as XLSX and PDF.
Prioritized requirements	Functional and non-functional requirements are documented in Section 4.
Design documents and UML	Architecture, UML, ER and flowchart diagrams are included.
Installation instructions	The setup is explained in Section 7 and in the README.
Executed test protocol	The test protocol is included as DOCX and PDF.

10. Conclusion

This project fulfills the M122E assignment because it automates a real weekly task with several steps and several systems. It is not only a small script that prints data. It loads real listening history, enriches it, stores it, calculates statistics, creates a styled report and can deliver the report by email.

The most valuable part for me was learning how many small details matter in an automation project. The code itself is important, but configuration, error handling, testing and documentation decide whether the project is actually usable. I also learned that optional features should not break the main workflow. The Spotify playlist update is a good example: it is useful when permissions work, but the weekly report must still be created even if the playlist update fails.

I am satisfied with the final result because the project is modular, tested and documented. The README helps someone start the project, the final documentation explains the design and the PDFs make the submission easy to read on GitHub. If I continued the project later, I would improve the playlist permissions, add more historical trends and possibly add a small web dashboard. For this module, the current version covers the required automation task well and shows the complete workflow clearly.

Looking back, my work process was very iterative. I first built the main automation path, then tested it with real Last.fm data, fixed Spotify permission issues, improved the email layout for desktop and mobile, and finally cleaned up the documentation and diagrams. This made the project feel much closer to a real automation task than a simple school script, because every improvement had to keep the weekly workflow stable.